A Comparative Empirical Study of FIDO2 and Its Predecessors

Anderko Máté
Óbuda University
mate.anderko@stud.uni-obuda.hu

Abstract—The FIDO2 standard, introduced by the FIDO Alliance and the World Wide Web Consortium (W3C) in 2018, it has been hailed as the "Kingslayer of User Authentication". This paper presents the implementation of a FIDO2 + WebAuthn authentication system and shows an analysis on different user authentication philosophies The analysis assesses a FIDO2, a 2FA and a password only system on their resistance to common security threats using the STRIDE threat modeling framework and evaluates performance based on latency, memory usage, and computational overhead. The objective is to determine how adopting FIDO2 influences the security, usability, and system resource requirements of information systems.

Index Terms—FIDO2, WebAuthn, Passkeys, Passwordless authentication, Multi-factor authentication, STRIDE, Performance

I. Introduction

Since the dawn of computing the weakest link in most information systems has been the people creating and using them. We choose easy to remember password that might be prone to dictionary attacks, then we overcomplicate them just to forget them, so to only have to remember a couple passwords we reuse them across sites, also we are susceptibile to social engineering attacks, like phishing. Sufficiently long, unique, and complex passwords with proper rate limiting remain impractical to brute-force, so compromises overwhelmingly exploit human factors and credential leaks rather than cryptographic weakness. Also the use passwords impose nontrivial support costs and friction because of the resets and account recovery.

To mitigate the weaknesses of a password only environment one-time codes (SMS/TOTP) and push prompts were introduced, this is safety standard is commonly referred to as Multi Factor Authentication MFA materially reduces risk from simple credential stuffing, but common forms remain phishable (e.g., OTP relay, look-alike pages) and are vulnerable to SIM-swap or "push fatigue" attacks. Usability trade-offs (codes, prompts, and device availability) also create drop-off during enrollment and sign-in.

FIDO2/WebAuthn addresses these limitations by replacing shared secrets with origin-bound public-key cryptography and local user verification (biometrics or a device PIN). Private keys never leave the authenticator; the server stores only public keys, eliminating password reuse and greatly reducing the value of stolen databases. Because assertions are tied to the requesting origin, generic phishing and OTP-relay attacks fail by design. In practice, passkeys (discoverable credentials) further streamline the flow across platforms using platform sync or roaming security keys.

This paper adopts that progression—passwords → MFA → FIDO2—and (i) implements a fully passwordless WebAuthn system, and (ii) compares password-only, password+MFA, and FIDO2 across resilience (phishing, stuffing, brute-force), usability (enrollment, recovery, cross-device), and resource usage (compute, storage, support). We show how MFA mitigates many password risks yet remains phishable, and how FIDO2 offers a principled, practical solution that is both more secure and simpler for end users.

II. Similar and Related Work

This section reviews research on the security, usability, and adoption of FIDO2, also shows a comparison with this here work.

A. Formal and Theoretical Analyses

Barbosa and their team [1] offers a formal model of FIDO2 and, through theoretical analysis, show resistance to credential-phishing and replay attacks. In contrast, the present work examines a deployed FIDO2 stack with STRIDE, emphasizing practical security implications and performance (latency, memory, compute).

B. Empirical and Usability Studies

Lyastani and their team [2] created an empirically FIDO2's usability analyses. The study shows significant usability gains and phishing resistance compared to passwords and 2FA. Their work relied on user studies and perception metrics rather than system-level measurements.

Unlike these human-centered, behavioral studies, this work isolates the technical performance and security aspects of FIDO2.

C. Positioning of This Work

Key contributions:

1) Operational implementation of FIDO2/WebAuthn alongside password-only and 2FA baselines.
2) Structured STRIDE analysis, grounded in Barbosa's theoretical model.

3) Empirical performance results—latency, memory footprint, computational load—that complement usability-centric studies (e.g., Lyastani).

Unlike work centered on formal guarantees or user experience, this study examines how adopting FIDO2 shifts real-world security posture and runtime efficiency.

III. Common Authentication Vulnerabilities

Bad actors find the most creative ways to attack authentication systems, these are called attack vectors. The goal of this section is to outline how the vulnerabilities work, which the paper will address in later sections, and also to shine a ligh on why traditional password or even basic MFA schemes struggle against them.s

A. Weak Passwords and Brute-Force Attacks

Users often choose simple passwords, making them easy targets for brute-force attacks [3]. A simple brute-force attack tries to crack the user's password by trying random character combinations. Rate limiting and sufficient password length makes the attack redundant. A dictionary attack is a more sophisticated version of a brute-force attack, where attackers use common wordlists, lists of frequenly used passwords or they systematically guess combinations to find the password. In the case of brute-force not cryptographic weaknesses, but bad password hygiene and the site's lack of password complexity enforcement leads to the compromise,.

B. Credential Stuffing and Password Reuse

Credential stuffing exploits password reuse across websites. In the event of a databreach the user's password may get leaked, this creates an opportunity for bad actors, who then use the credentials on other sites to try to gain access to other accounts.[4]. The process can be well automated, thus the reuse of credentials turns one breach into many.

C. Social Engineering Attacks

Many attacks rely on human gullability instead of technical flaws. Phishing, SIM swap, and push fatigue attacks are such attacks.

1) Phishing and OTP Relay: Phishing deceives users into revealing credentials on fake websites, that mimic the working of the site the user is familiar with. Attackers can proxy the login and capture OTP codes in real time, bypassing traditional 2FA, this is commonly referred to as a real-time phishing proxy. OTP capturing is Adversary-in-the-Middle type attack[5].

2) SIM Swap Attacks: The way SMS based MFA works is that the user gets an SMS containing a secret code, generated by the server, this while may seems bulletproof requires complete trust in the user's telecommunication service provider. The attackers may be able to impersonate victims and hijack their phone number (with the service provider unknowingly complicit), thus gaining control of SMS-based OTPs[6].

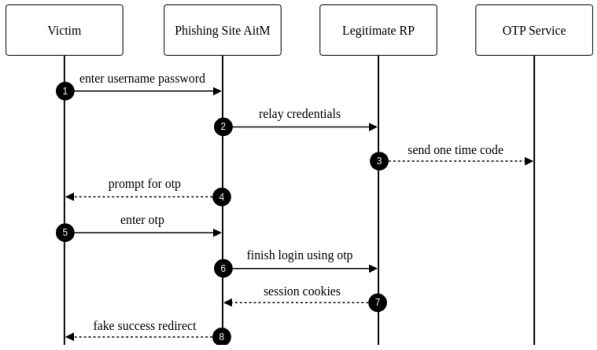

Fig. 1. Phishing with OTP relay

3) MFA Push Fatigue: Push-based MFA can be exploited by flooding users with repeated prompts until they approve one, often by mistake or frustration [7].

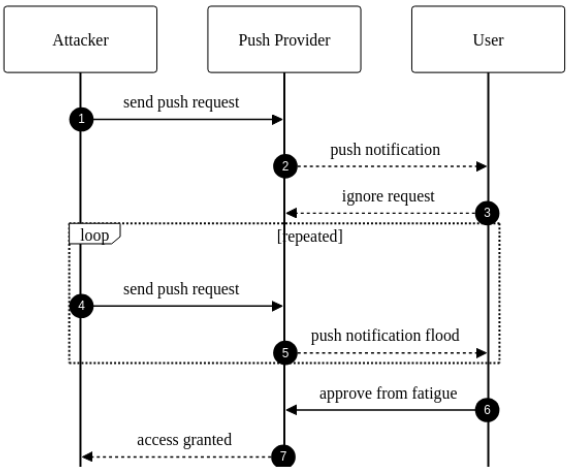

Fig. 2. Push fatigue attack sequence

IV. How FIDO2 Works

This section offers a quick introduction into the FIDO2 ecosystem. FIDO2 provides an phishing-resistant, origin-bound public-key authentication by combining the W3C WebAuthn API (browser/platform) with the FIDO Alliance CTAP protocol.

A. Core Roles

1) Relying Party (RP): The application service that issues authentication challenges and stores each user's registered public key.
2) Client/Platform: The browser or operating system that brokers WebAuthn requests and responses between the RP and the authenticator.
3) Authenticator: A platform or roaming device that creates key pairs, enforces user presence/verification, and returns attested assertions.

B. Registration (makeCredential)

1) The RP provides options (challenge, RP ID, user handle, algorithm preferences) to the client.
2) The authenticator creates a new key pair based on the RP ID, the private key remains within the authenticator, keeping it safe from attackers, the public key and attestation statement are returned.
3) The RP verifies attestation and stores the credential public key.

C. Authentication (getAssertion)

1) The RP provides options (challenge, RP ID, and an allow list if applicable).
2) The authenticator enforces user presence/verification, then signs the challenge together with the RP ID hash, authenticator flags, and a signature counter.
3) The RP verifies origin, RP ID, signature, and counter before completing authentication.

D. Security Properties

- Origin binding (via RP ID and client data) mitigates phishing and relay.
- Non-exportable keys reduce the impact of server-side compromise.
- Attestation enables enforcement of device-class policy, lower class untrusted devices can be blocked.

E. Real-World example

This example tries to convey how Android devices keep the data secure working with the FIDO2 ecosystem. On Android, passkeys and cryptographic keys are managed through Credential Manager and the Android Keystore. Where available, StrongBox places KeyMint in a discrete secure element so that keys are generated and stored within a dedicated secure processor rather than solely in the TEE. Private key material is non-exportable and invocable only via Keystore operations under enforced user-authentication and policy.

1) StrongBox isolation: StrongBox uses KeyMint in a separate secure chip. This chip has its own CPU,RAM and secure storage, completely isolating it from the mobile device, thus making it virtually impossible for bad-actors to access the private key stored here. I enables hardware attestation so servers can verify that keys are hardware-backed on a genuine device.

2) Example Flow:

- The registration process begins through the Credential Manager (WebAuthn).
- The Keystore requests a hardware-backed key; on devices with StrongBox, the key pair is generated within the StrongBox KeyMint and marked as non-exportable.
- The public key and attestation chain are returned. The RP may verify the attestation to confirm hardware-backed security.
- During authentication, StrongBox performs the signature after local user verification; the private key never leaves the secure processor.

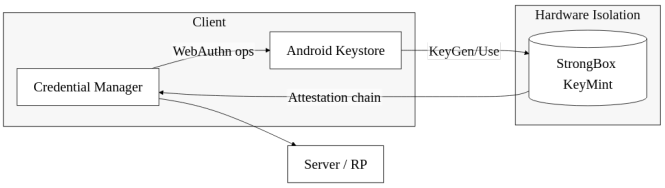

Fig. 3. Android StrongBox flow

V. STRIDE Threat Modelling Framework

The STRIDE threat modeling framework helps to group real-world attacks into six types. Each type targets a different part of the authentication process. This section maps the vulnerabilities discussed earlier to STRIDE categories and shows how each of the three authentication methods handles them.

STRIDE categories:

- Spoofing: when an attacker pretends to be a valid user to gain access
- Tampering: when an attacker changes messages or data during login
- Repudiation: when a user denies doing an action, like logging in
- Information Disclosure: when sensitive data like passwords or OTPs are leaked
- Denial of Service: when the attacker blocks the user from logging in
- Elevation of Privilege: when someone gets more access than they should

Each vulnerability from Section III fits one or more of these categories. Table III (see Section VIII) shows which threat is mapped where, and what to expect from password-only, 2FA, and FIDO2-based systems.

The STRIDE mapping helps compare which attack types are stopped or remain possible in each authentication setup. This forms the basis of the analysis in Section ??, where each system's real threat resistance is examined in more detail.

VI. Implementation

This section documents the packages used and how components interact in the prototype.

A. Backend Packages

The backend is built with FastAPI and uses the packages listed in Table II (see Section VIII). FastAPI provides asynchronous request handling and automatic OpenAPI documentation. The 'fido2' library (v2.0) handles WebAuthn protocol operations, including challenge

generation, attestation verification, and assertion validation. SQLAlchemy serves as the ORM layer connecting to PostgreSQL, while Redis stores ephemeral state such as WebAuthn challenges and password reset tokens.

B. Component Interactions

The backend routes HTTP requests through dedicated routers (‘fido‘, ‘password‘, ‘totp‘) mounted under ‘/api/v1‘. Each router queries the unified ‘Credential‘ model via SQLAlchemy, which persists data to PostgreSQL. Ephemeral state (WebAuthn challenges, password reset tokens) is stored in Redis with namespaced keys. Successful authentication returns a JWT token issued by ‘pyjwt‘ for session management.

VII. Performance Testing

This section presents empirical latency measurements comparing FIDO2, password-only, and TOTP authentication methods. The tests measure end-to-end authentication flows to provide a fair comparison of user-perceived performance.

A. Test Methodology

1) Test Framework and Setup: The latency measurements were conducted using a custom testing framework built on pytest with asynchronous HTTP client support. Tests run against an in-memory SQLite database and a fake Redis instance to ensure isolation and reproducibility. Each test executes multiple iterations (typically 10) to gather statistically significant data.

2) Measurement Approach: Latency is measured using high-precision timing utilities based on Python's time.perf_counter(), which provides microsecond-level accuracy. Measurements are captured using context managers that wrap each operation, ensuring consistent timing boundaries. The framework measures:

- Endpoint latency: Total time for HTTP request-response cycle
- Operation breakdown: Individual steps within complex flows
- End-to-end latency: Complete user authentication experience

3) Test Scenarios: Password Authentication:

- Registration: Single API call to create user and hash password (bcrypt)
- Login: Single API call verifying password hash and issuing JWT token

TOTP Authentication:

- Setup: Password verification + TOTP secret generation + QR code creation
- Login: Password verification + TOTP code validation (HMAC-SHA1) + token issuance

FIDO2 Authentication:

- Registration: Three-step flow:
  1) /register/start: Server generates challenge and credential options
  2) WebAuthn create: Authenticator creates key pair and attestation (simulated with 100ms delay)
  3) /register/finish: Server verifies attestation and stores credential
- Login: Two-step flow:
  1) /login/start: Server generates authentication challenge
  2) /login/finish: Authenticator signs challenge, server verifies signature

4) Fairness Considerations: To ensure fair comparison despite different authentication complexities, measurements are taken at multiple levels:

1) Backend processing time: Isolated server-side operations
2) Network round-trips: Number of HTTP requests required
3) End-to-end flow time: Total user experience (what matters most)
4) Cryptographic overhead: Time spent on crypto operations (bcrypt, HMAC, ECDSA)

The FIDO2 authenticator interaction is simulated with a 100ms delay, representing a typical hardware authenticator response time. This value is based on empirical observations of common authenticator devices (USB security keys, platform authenticators) which typically require 50-200ms for:

- Key pair generation or signature computation
- User presence/verification (if required)
- Communication with the browser/platform

The 100ms value represents a mid-range estimate, allowing measurement of server-side processing while acknowledging that real-world FIDO2 performance varies significantly based on authenticator hardware (hardware security keys vs. platform authenticators vs. software authenticators).

B. Results

Table I presents the mean latency measurements for each authentication method. All measurements are in milliseconds, based on 10 iterations per test.

TABLE I
Latency Measurements Summary (mean values in milliseconds)

Operation	Method	Mean Latency (ms)
Registration/Setup	Password	216.3
Registration/Setup	FIDO2	108.2
Registration/Setup	TOTP	227.8
Login	Password	216.4
Login	TOTP	216.1
Login	FIDO2	102.1

Figure 4 compares registration latencies. FIDO2 registration (108.2 ms) is significantly faster than password

registration (216.3 ms), despite requiring two network round-trips versus one. This is because password registration includes expensive bcrypt hashing (typically 100-200ms), while FIDO2 registration delegates cryptographic operations to the authenticator. TOTP setup (227.8 ms) is the slowest, as it requires both password registration (bcrypt hashing) and additional TOTP secret generation, backup code creation, and QR code generation.

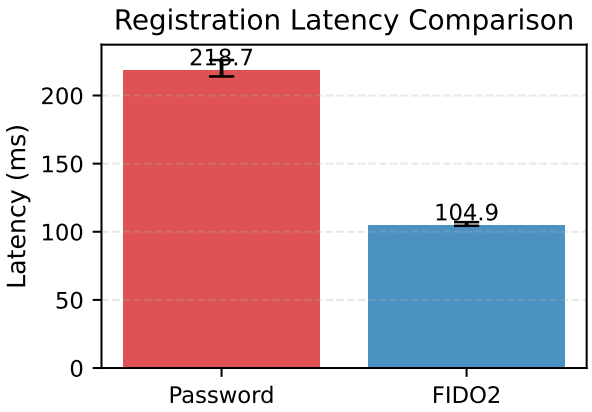

Fig. 4. Registration latency comparison between Password and FIDO2 authentication methods. Error bars show min-max range.

Figure 5 shows login latencies. Password and TOTP logins are nearly identical (205ms) since both require bcrypt password verification. FIDO2 login (102ms) is faster than password/TOTP, with the majority of time (100ms) spent in authenticator interaction (signature generation and user verification). The server-side operations (challenge generation and signature verification) are minimal (2ms), demonstrating FIDO2's efficiency once credentials are registered.

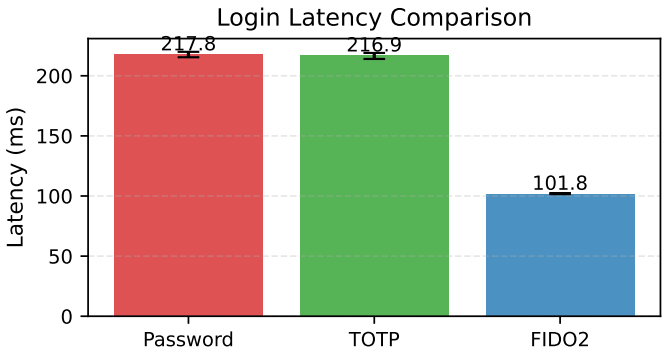

Fig. 5. Login latency comparison across all three authentication methods. Error bars show min-max range.

Figure 6 breaks down FIDO2 registration into its components. The WebAuthn authenticator interaction (simulated at 100ms) dominates the total time, while server-side operations (start: 4.2ms, finish: 1.9ms) are minimal.

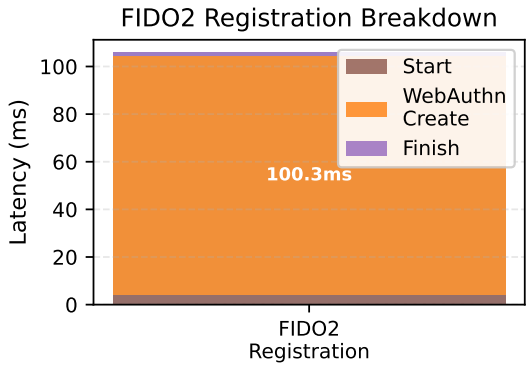

Fig. 6. FIDO2 registration latency breakdown showing time spent in each phase.

C. Analysis and Discussion

1) Key Findings: 1. Registration/Setup Performance: FIDO2 registration (108ms) is faster than password registration (216ms) despite requiring two network requests. The difference stems from bcrypt hashing overhead in password registration. FIDO2 offloads cryptographic key generation to the authenticator, reducing server CPU load. TOTP setup (228ms) is the slowest, requiring password registration plus TOTP secret generation, backup code hashing, and QR code image generation. The QR code generation adds 10-20ms overhead compared to password-only registration.

2. Login Performance: FIDO2 login (102ms) is faster than password/TOTP (205ms), with the authenticator interaction (100ms) being the dominant component. The server-side operations are minimal:

- Challenge generation (2ms)
- Authenticator signature generation (100ms, includes user verification)
- ECDSA signature verification (1ms)
- Database credential lookup

The authenticator delay represents hardware security key or platform authenticator response time, which varies by device but is typically 50-200ms.

Password and TOTP logins both require bcrypt verification, explaining their similar latencies (216ms). TOTP adds minimal overhead: HMAC-SHA1 computation for code verification takes approximately 0.03ms, which is negligible compared to bcrypt password hashing (200ms). The TOTP validation overhead is less than 0.02% of the total login time, making it effectively invisible in end-to-end measurements.

3. Network Round-Trips: FIDO2 requires two HTTP requests (start + finish) versus one for password/TOTP. However, this architectural difference does not significantly impact total latency in our measurements, as

network latency is minimal in the test environment. In real-world scenarios with higher network latency, the two-request pattern may have more impact.

4. Security-Performance Trade-off: FIDO2 provides superior security (phishing-resistant, origin-bound credentials) with acceptable performance overhead during registration. Once registered, FIDO2 offers both better security and better performance than password-based methods.

2) Limitations: The measurements have several limitations:

- Mocked authenticator: FIDO2 tests use simulated authenticator responses. Real hardware authenticators may have different latencies (typically 50-200ms for user verification).
- Local testing: Network latency is minimal in test environment. Real-world deployments may see different relative performance.
- Single server: Tests run on a single machine. Distributed systems may show different characteristics.
- bcrypt cost factor: Password hashing time depends on bcrypt cost factor. Our implementation uses default settings; higher security requirements would increase password/TOTP latency.

3) Implications: For system designers choosing authentication methods:

1) FIDO2 offers the best combination of security and performance, especially for login operations. The registration overhead (100ms authenticator interaction) is a one-time cost that pays dividends in faster, more secure logins.
2) Password-only authentication has the highest latency due to bcrypt hashing, with no security benefits over FIDO2.
3) TOTP provides two-factor security but offers no performance advantage over password-only, as both require bcrypt verification. The additional TOTP validation (HMAC-SHA1) adds negligible overhead (0.03ms), which is less than 0.02% of the total login time. The bcrypt password verification (200ms) completely dominates the latency.

These results support FIDO2 adoption for systems prioritizing both security and user experience, particularly for frequently-used authentication flows where login performance matters most.

VIII. Tables

This section contains all tables referenced throughout the document.

References

[1] M. Barbosa et al., "Provable security analysis of fido2," in Information Security Theory and Practice (WISTP 2021). Springer, 2021. [Online]. Available: https://dl.acm.org/doi/10.1007/978-3-030-84252-9_5

[2] S. Lyastani et al., "Is fido2 the kingslayer of user authentication?" in IEEE Symposium on Security and Privacy, 2020. [Online]. Available: https://trust.cispa.saarland/publication/lyastani-20-sp/lyastani-20-sp.pdf

[3] E. Ma, S. Hasama, E. Lumba, and E. Birrell, "Prospects for improving password selection," arXiv preprint arXiv:2201.01350, 2022.

[4] A. Nisenoff, M. Golla, M. Wei, J. Hainline, H. Szymanek, A. Braun, A. Hildebrandt, B. Christensen, D. Langenberg, and B. Ur, "A two-decade retrospective analysis of a university's vulnerability to attacks exploiting reused passwords," in Proc. USENIX Security Symposium, 2023.

[5] M. Käsemodel, V. Winterhalter, F. Heisel, F. Alt, and B. Pfleging, "BlueTOTP: Designing phishing-resistant and user-friendly two-factor authentication," in Proc. IFIP INTERACT 2025 (LNCS 16110). Springer, 2025, pp. 611–634.

[6] K. Lee, B. Kaiser, J. Mayer, and A. Narayanan, "An empirical study of wireless carrier authentication for SIM swaps," in Proc. 16th Symposium on Usable Privacy and Security (SOUPS). USENIX Association, 2020, pp. 61–79.

[7] Cybersecurity & Infrastructure Security Agency (CISA), "Implementing phishing-resistant mfa (fact sheet)," https://www.cisa.gov/sites/default/files/publications/fact-sheet-implementing-phishing-resistant-mfa.pdf, 2022, accessed 2025-10-29.

TABLE II
Backend Python packages and their roles

Package	Role
‘fastapi‘	Web framework providing REST API endpoints and request routing
‘uvicorn‘	ASGI server running the FastAPI application
‘fido2‘	WebAuthn protocol implementation for credential registration and authentication
‘sqlalchemy‘	ORM for database operations and model definitions
‘psycopg2-binary‘	PostgreSQL database adapter
‘pydantic‘	Data validation and settings management
‘redis‘	State store for temporary challenge data
‘pyjwt‘	JWT token generation and verification
‘cbor2‘	CBOR encoding/decoding for FIDO2 binary data
‘bcrypt‘	Password hashing with configurable work factor
‘pyotp‘	TOTP code generation and verification
‘qrcode‘	QR code generation for TOTP secret enrollment

TABLE III
STRIDE Categories, Real-World Vulnerabilities, and Expected Behavior

STRIDE Category	Mapped Vulnerability	Password-only	2FA	FIDO2/WebAuthn
Spoofing	Phishing and OTP relay attacks	Easy to spoof if password is stolen or phished	OTP relay attacks can still trick users	Origin check and public-key signatures prevent spoofing
Tampering	Adversary-in-the-middle modifying login flow	No strong protection if TLS fails	Relayed OTPs or injected prompts possible	Signed challenge-response blocks message tampering
Repudiation	No way to prove who logged in	Anyone with the password can log in	2FA logs may help but are weak proof	Authenticator signs each login, tied to device and origin
Information Disclosure	Password reuse and data breaches	Password leaks are common and reused across sites	OTP seeds and SMS codes can be stolen	No password or shared secret stored or sent
Denial of Service	Push fatigue and SIM swap	Lockout by brute force or repeated failures	Flood of OTP or push requests can lock out users	Brute-force blocked by design, backup keys help recovery
Elevation of Privilege	Use of weak or reused admin credentials	Admin access can be guessed or reused	OTP can be phished from high-priv users	High-priv accounts tied to secure device credentials